CONF42 PLATFORM ENGINEERING

Before You Pick an Agent Orchestrator

What I learned wiring a self-driving SDLC



Yedidya Schwartz
CTO, Quicklizard

Five parts

01 **The big picture**
AI SDLC: why faster coding alone is not enough

02 **The first try**
From copy-paste glue to one big prompt, and where it broke

03 **Choosing the engine**
Why I moved the plan into code, and why Temporal

04 **What broke**
War stories, and the lessons they taught me

05 **Is it worth it?**
What Stripe and Spotify learned, and where this is going

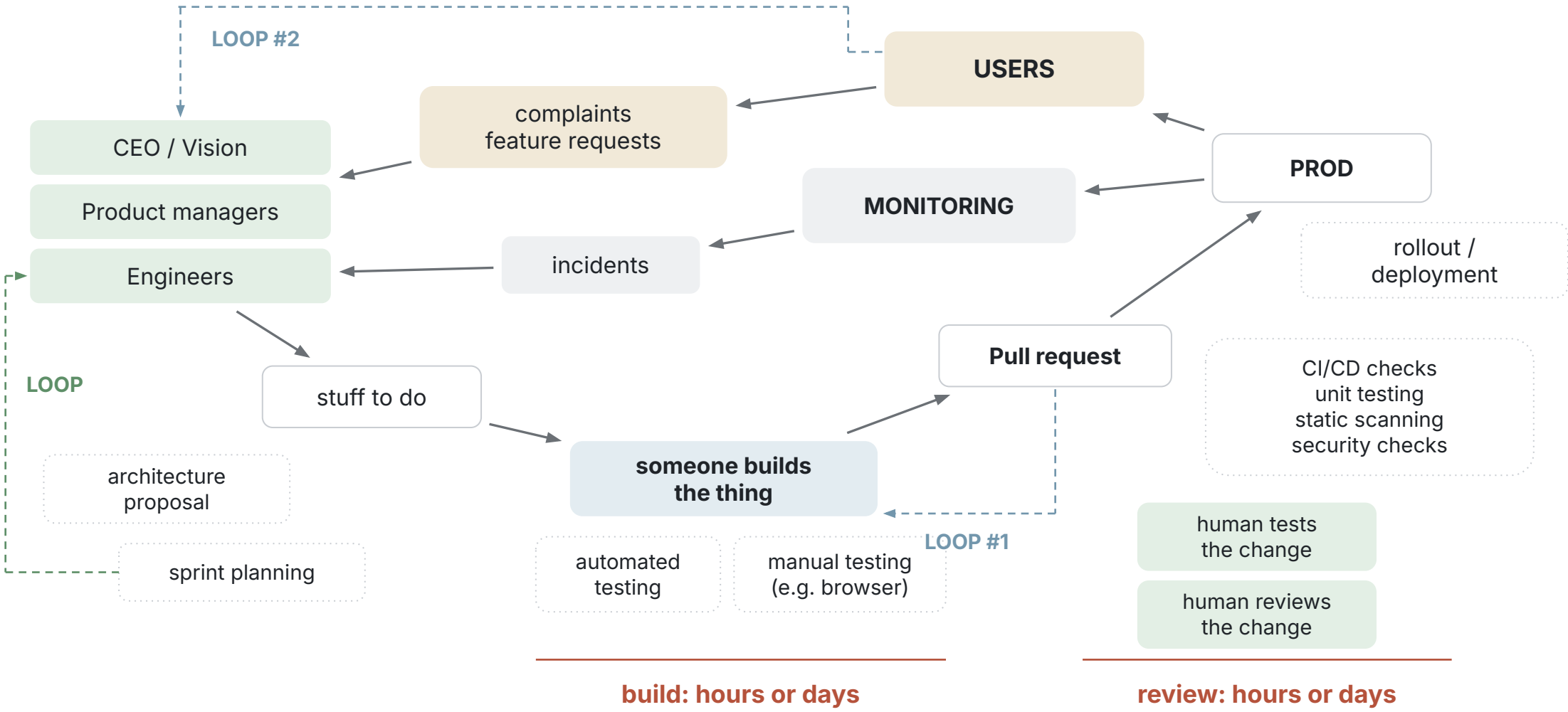
01

PART ONE OF FIVE

The big picture

AI SDLC: why faster coding alone is not enough

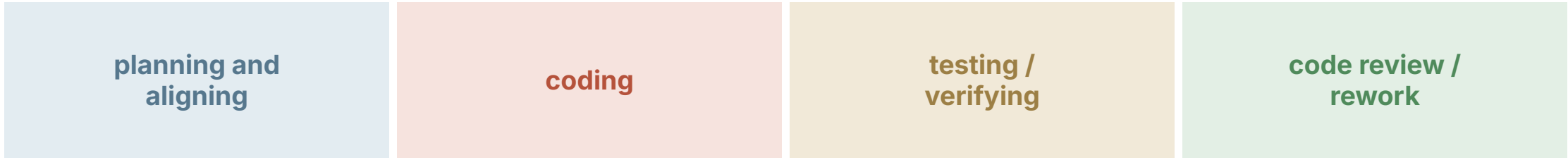
A Software Factory, before agents



Source: Dex Horthy · Why Software Factories Fail

Before AI

“It's a two day feature. The coding will take 2–4 hours.”



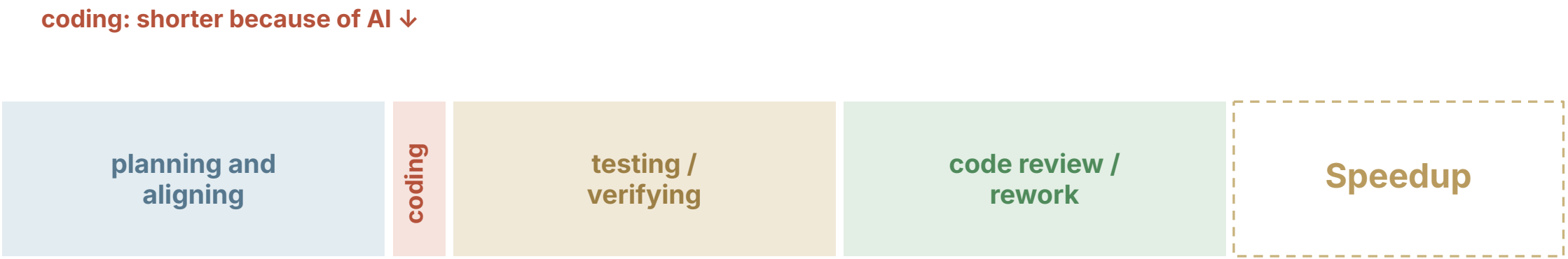
the time a two-day feature took before AI

Coding is one quarter of the work. The rest is everything around it.

Source: Dex Horthy · Why Software Factories Fail

Naive AI

“It's a two day feature. The coding will take 20 minutes.”

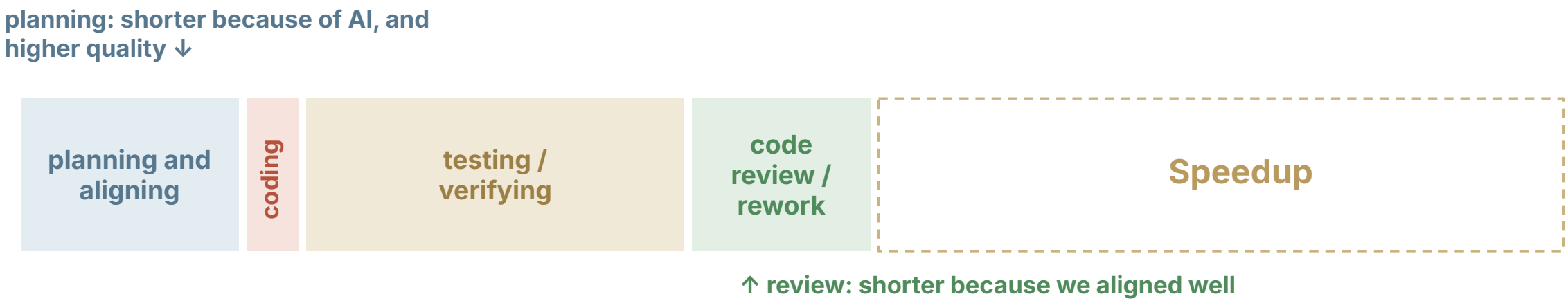


Coding got much faster. Planning, testing and review still take the same time.

Source: Dex Horthy · Why Software Factories Fail

Model-assisted planning and alignment

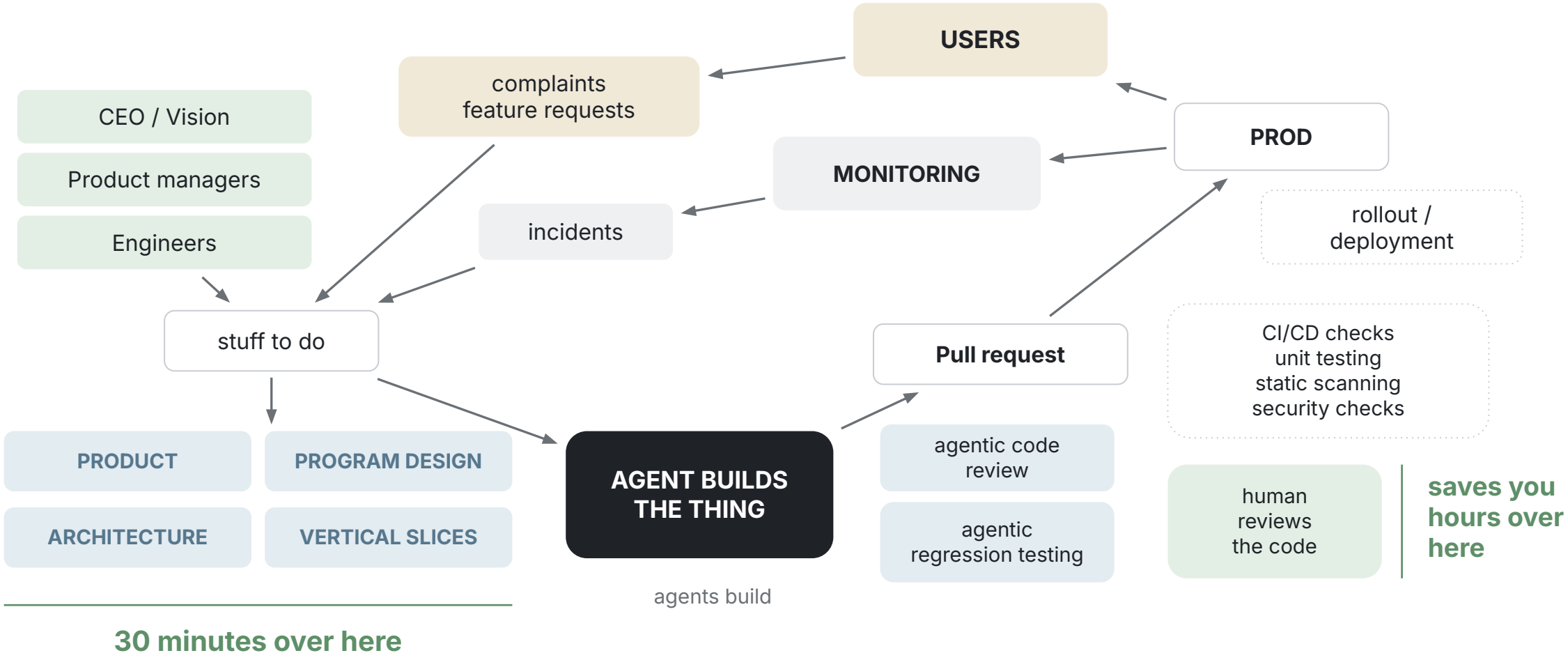
Use the model to plan and align, not only to code.



Better planning cuts review and rework too. That's where the real time goes.

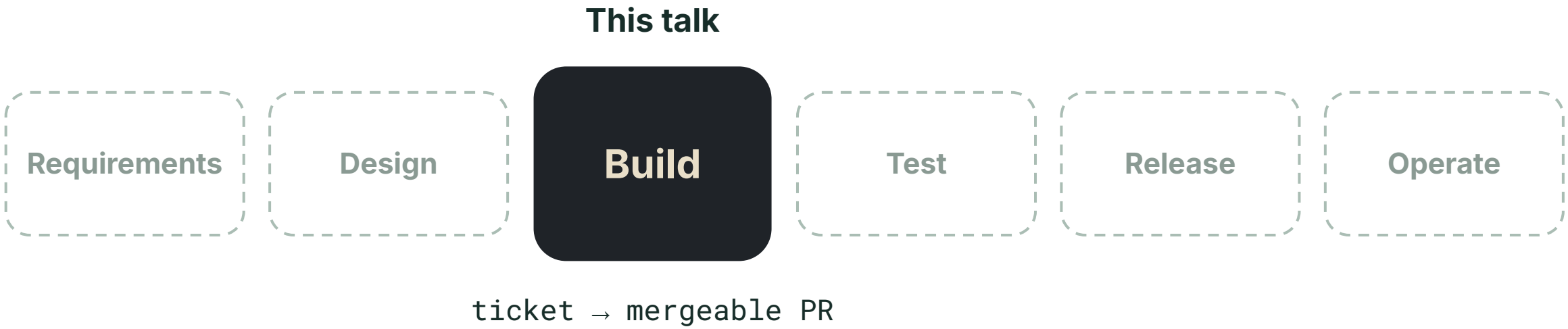
Source: Dex Horthy · Why Software Factories Fail

Optimizing the software factory with agents



Source: Dex Horthy · Why Software Factories Fail

This talk zooms in on one stage: Build



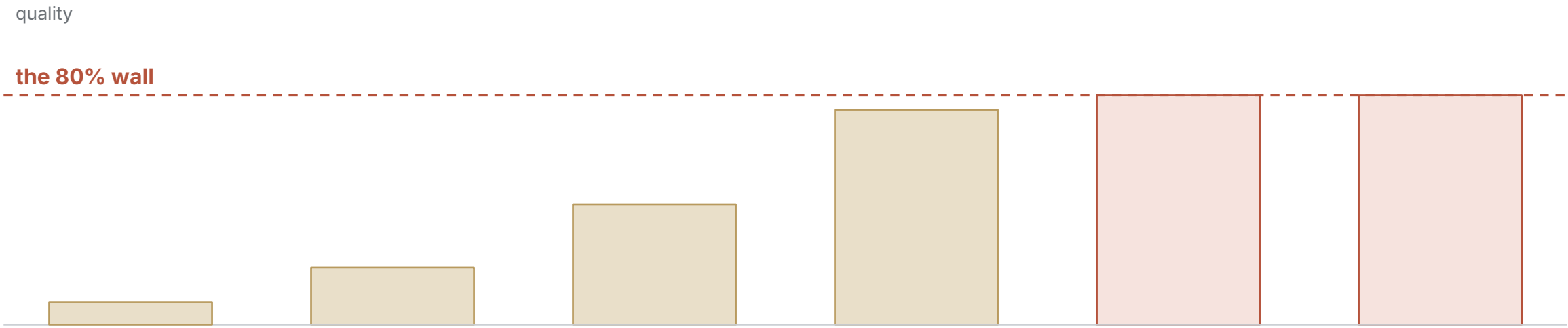
Before: deciding what to build

After: proving, shipping and running it

Source: IBM · AI in the SDLC (video)

THE BIG PICTURE · DEX HORTHY

The agent journey



1. Decide you want to build an agent
2. Product design, UX mapping, what problems to solve
3. Want to move fast, so grab $FRAMEWORK and get to building
4. Get to 70-80% quality bar
5. **Realize that 80% isn't good enough**
6. **Realize that getting past 80% requires reverse engineering the framework, prompts, flow, etc.**

Source: Dex Horthy · 12-Factor Agents

02

PART TWO OF FIVE

The first try

From copy-paste glue to one big prompt, and where it broke

What would it take to turn a ticket into a PR I could merge?

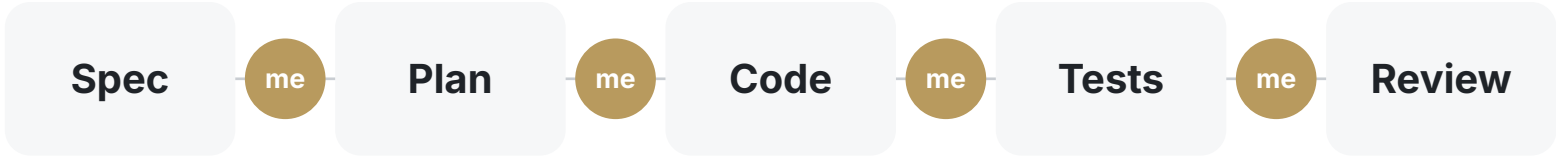
THE FIRST TRY

One feature built by hand, and I was only the glue

4 days

of me in the middle

An admin screen built with Claude Code, step by step. Every hand-off between the steps went through me.



Between every two steps, the same three moves:

COPY

Take the output of one step.

PASTE

Feed it into the next one.

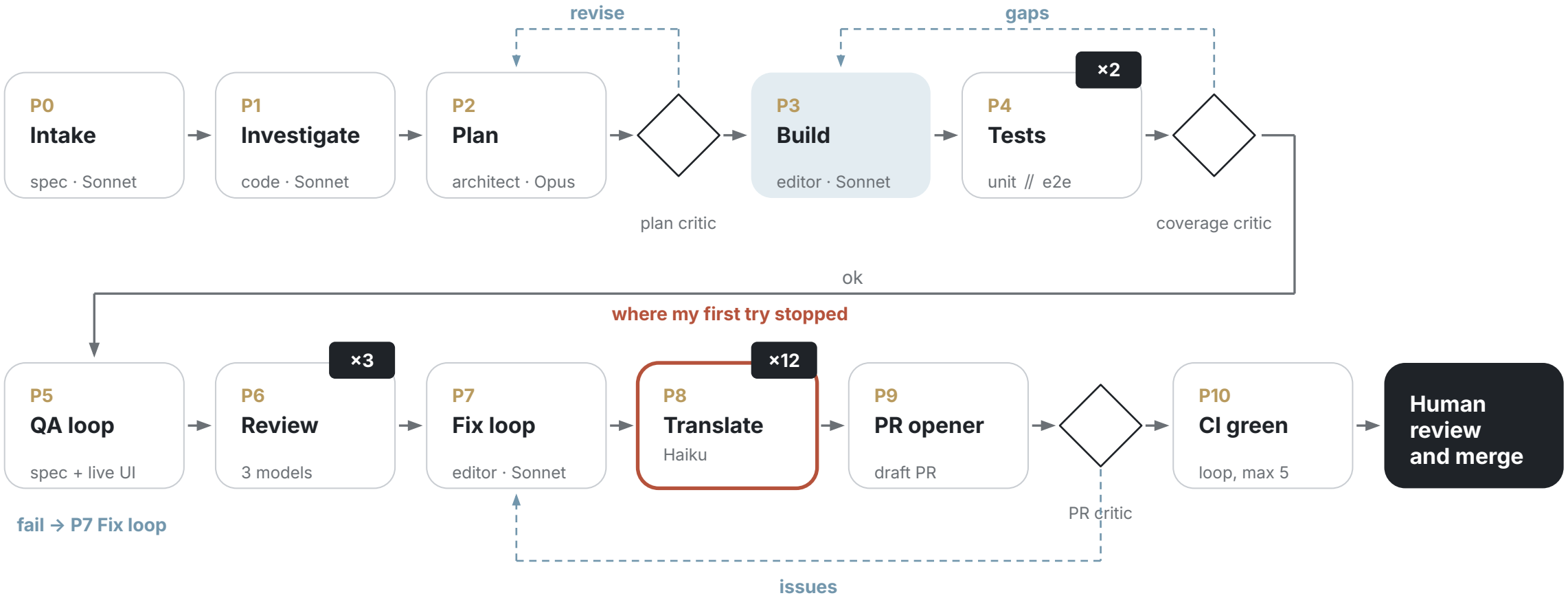
CHECK

Pass or fail? Then on to the next step.

That isn't engineering judgment. It's a job for code.

So why shouldn't this run without me?

The pipeline I wanted: eleven phases, from ticket to PR



THE FIRST TRY

What's the simplest thing that could possibly work?

That was my only design rule. No platform, no infrastructure. Build the simplest thing that runs, and let it show me where it hurts.

THE FIRST TRY

One prompt on a GitHub runner

Me
push a prompt file, click Run

→

GitHub Actions
a manual workflow run

→

Self-hosted runner
a pod in the dev cluster

→

Claude Code
one 338-line prompt runs P0 to P10

→

Draft PR
on my branch

GitHub shows: running, then success or failed

WHY IT LOOKED PERFECT

A machine on demand

The runner starts a pod for each run. No new servers to run.

Status for free

GitHub shows running, success, failed. That was my whole dashboard.

Already inside

The runner lives in the dev cluster, so it reaches our services.

Zero new infra

No queue, no database. The state was a JSON file in the branch.

WHAT IT DIDN'T HAVE

- Only the model knew where it was
- No recovery when the pod dies
- “Success” only meant the process exited

I wanted to see where it gets hard, before building anything hard.

WHERE IT BROKE

The job was green. The pipeline had stopped at P8.

One prompt · 12+ subagents · 2 hours 20 minutes

What GitHub sees — *it checks the exit code, not the work* — ✔ success

INSIDE THE RUNNER POD · **deleted when the job ended**

The model, running one long prompt
it decides which phase runs next

"Here's a summary of the i18n work..."
1,800 words. Then it stopped.

P0 Intake	P1 Investigate	P2 Plan	P3 Build	P4 Tests	P5 QA	P6 Review	P7 Fix	P8 Translate	P9 PR	P10 CI

P0 to P7 worked. QA passed 34 of 34 checks. The review caught a real bug.

never ran

POD DELETED
all the work was on its disk

NOTHING PUSHED
the commits never left the pod

ALREADY PAID
every model call, for nothing

NO TRACES
nothing to debug with

Only the model knew three phases were left, and it decided it was done.
The fix was not a better prompt. It was moving that knowledge into code.

Source: Cemri et al., Why Do Multi-Agent LLM Systems Fail? (UC Berkeley, 2025)

Dex Horthy's 12-Factor Agents, in five lines

1. Agents are Software
2. Everything is Context Engineering
3. **Own your state and control flow**
4. Find the bleeding edge
5. Agents are better with people

TWO QUESTIONS FOR THE REST OF THIS TALK

1 Who decides what runs next?

2 Who checks that the work is really done?

Source: Dex Horthy · 12-Factor Agents

03

PART THREE OF FIVE

Choosing the engine

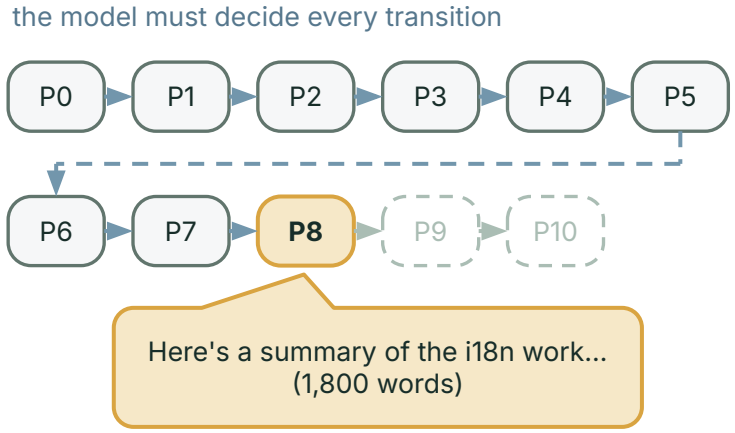
Why I moved the plan into code, and why Temporal

Same pipeline, two brains: a prompt vs. a for-loop

POC: the LLM is the orchestrator

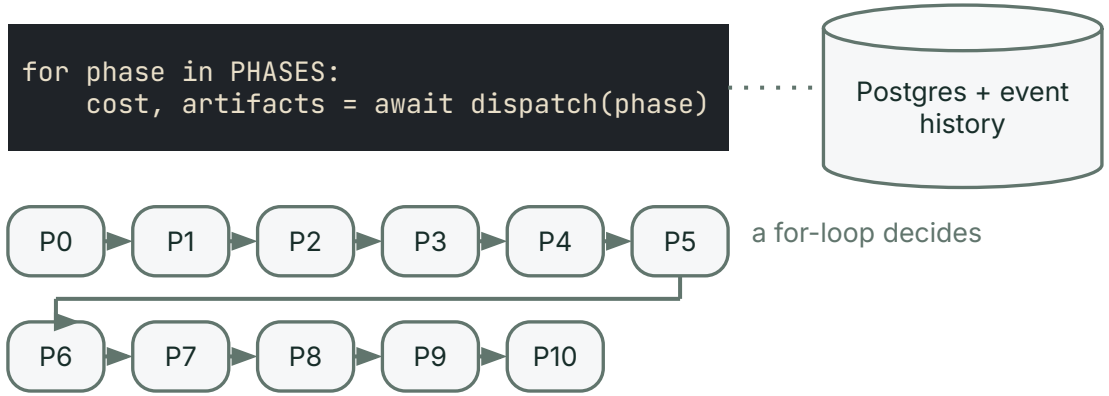
```
# state machine
## P0 intake
## P2 plan
…
338 lines
```

state.json committed to the branch



Code is the orchestrator (Python on Temporal)

```
for phase in PHASES:
    cost, artifacts = await dispatch(phase)
```



	POC: Markdown prompt	Code: Python workflow
State machine	338-line Markdown prompt	Python, registry-driven
Phase advancement	the model must remember to dispatch	`for phase in PHASES`
Recovery	none, ephemeral runner pod	replay from history
Retries	every retry re-spends tokens	skip if the output is already saved
Stopping early (like at P8)	open	can't happen: it's a for-loop
Testing the orchestrator	run the whole 2h workflow	fake-clock test: 0.5 s

CHOOSING THE ENGINE

The spec I was shopping for

THE HARD CONSTRAINT

It must run inside my private network.

Agents build and test against our internal services, databases and infra. No VPC access, no job.

Steps of 20 minutes to 4 hours

One agent session per phase. Long, silent, expensive.

A pod dies in the middle

Deploys, spot nodes, out of memory. Go on from where it stopped.

Never pay twice

A finished phase cost real tokens. A restart must not run it again.

Loops and gates

A critic says REVISE and the flow goes back. Not a straight line.

Wait for a human

Approve the plan, then continue. Maybe hours later.

See every step

What ran, what retried, what it cost, and why it stopped.

A prompt can't promise any of this. A hosted sandbox can't reach my network.

Three different jobs all get called “orchestration”

DAG schedulers

Airflow, Prefect

✓ Run a graph of jobs on a schedule.

✕ Not built for replay per step or waiting days for a human.

Agent frameworks

LangGraph and friends

✓ The agent loop: streaming, memory, tool routing, evals.

✕ Little infrastructure, but one process. A crash mid-node starts over.

Durable execution engines

Temporal, Restate, DBOS…

✓ Survive a crash, resume exactly, retry one step, wait days for a signal.

✕ You build streaming, memory and prompt management yourself.

A common, sane pattern: a durable engine for the long-running flow, and an agent framework for the reasoning loop inside one step.

CHOOSING THE ENGINE

Everything I tried, and the limit that ruled it out

OPTION	WHAT RULED IT OUT FOR MY PIPELINE	RUNS IN MY VPC?
One big prompt on a CI runner	The model decides when it's done. It stopped mid-pipeline, and the job still said success.	**Yes**
LangGraph	Checkpoints sit between nodes. A pod dies 3 hours into a node: the whole node runs again.	Library yes. Server: Enterprise
Airflow · Dagster · Argo	One-way graphs. Sending work back to a critic is a workaround, and a 4-hour task retries from zero.	**Yes**
Prefect	A crashed run is just marked Crashed. A human retries it, and pays for the phase again.	**Yes**
Step Functions	JSON states, 256 KB per state, 25,000 history events. REVISE loops eat the cap.	Engine in AWS, workers in VPC
Inngest · Trigger.dev	Inngest caps a step at 2 hours. Self-hosted Trigger.dev drops its checkpoints.	Self-hosted, fewer features
DBOS	The honest middle ground. But a deploy can strand runs, and cross-pod recovery is paid.	**Yes**
Restate · Hatchet · Dapr	Strong engines, but the defaults kill a silent call: Restate after ~11 quiet minutes, Hatchet after 60 s.	**Yes**
Temporal	**Kept. Finished phases never rerun. The price: I run the database and tune the timeouts.**	**Yes**

CHOOSING THE ENGINE

The defaults that kill a 4-hour agent step

TEMPORAL

No heartbeat timeout by default

A pod that dies 10 minutes into a 4-hour phase is noticed only when the whole timeout ends.

TEMPORAL

History dies at 50 MB

One team passed prompts through history. Their run was killed at model call 53 of 100.

RESTATE

Quiet 1 minute: suspend. 10 more: abort

A Claude session that thinks quietly gets killed, and paid for again.

HATCHET

60-second default timeout

Every long phase dies, unless you override the timeout on each task.

CONDUCTOR

No status for 600 s: requeue

The silent call is sent again while the first one still runs. Tokens spent twice.

LANGGRAPH SERVER

3-minute tool calls re-sent

Both copies finish, at 2 to 3 times the cost. The issue is open with 88 comments.

CLAUDE CODE HEADLESS

Idle 600 s: helpers killed

One run: 9 of 11 sub-agents killed, $110 spent, and the job exited with success.

COPILOT CODING AGENT

59-minute hard cap

Even on your own runners inside your VPC. One phase alone can run longer.

Most defaults assume a step takes seconds. An agent step can take hours. Check the defaults before you check the features.

CHOOSING THE ENGINE

My hard constraint ruled out the hosted agents

If it runs in the vendor's sandbox, it can't build and test against my internal services and databases.

- ✕ **Claude Code on the web** — Anthropic's VMs. In-network runners are a Team/Enterprise beta.
- ✕ **OpenAI Codex cloud** — OpenAI's VMs. Its Tailscale VPN is for HTTP services, not databases.
- ✕ **Google Jules** — A cloud VM per task. No private network option.
- ✕ **Kiro cloud sessions** — Git and internal services must be reachable from the internet.
- ✕ **Devin** — Private access only on Enterprise. Brain stays in Cognition's cloud.
- ✕ **LangGraph Cloud** — Hybrid and self-hosted are Enterprise only.
- ✕ **Vercel Workflow** — Private networking is Enterprise only, and caps a step at 800 s.
- ✕ **Cloudflare Workflows** — Private networking is beta. Claude Code needs a Sandbox.

None reaches my databases without Enterprise or a beta.
And none gives me durable phases, loops and gates.

CHOOSING THE ENGINE

Try fast, fail fast, change fast

Try fast

GitHub Actions: the simplest way to run a prompt on a remote machine inside my VPC.

Fail fast

It stopped after 8 of 11 phases and GitHub said success. The plan had to leave the prompt.

Change fast

An external workflow orchestrator. I picked Temporal, the one I knew well from past work.

WHY TEMPORAL

I knew it well

Experience from past work, so I could move fast from day one.

Read every run

The full history of each step, in a UI.

Never pay twice

Results come from history. Only the phase in flight runs again.

It met every requirement

Inside my VPC, hours-long steps, loops, human gates. All on my list.

The price: I run the database, tune heartbeats, and wrap the Claude SDK myself.



HOW TEMPORAL WORKS

Temporal in one slide: your code runs, the server remembers

WORKFLOW · the plan, as code

```
for phase in PHASES: run_phase(phase)
```

Decides what runs next. Must decide the same way every time.

Call the model (costs money)

Save the files

Run the real check

ACTIVITIES · the real work. Anything that touches the world. They can fail and retry on their own.

WORKER · your own process. It pulls tasks from a queue and runs the workflow and the activities. Kill it, and another worker takes over.

EVENT HISTORY · kept by the Temporal server

1	WorkflowExecutionStarted	
2	ActivityTaskScheduled	P0 intake
3	**ActivityTaskCompleted**	**P0 → spec saved**
4	ActivityTaskScheduled	P1 investigate
5	**ActivityTaskCompleted**	**P1 → notes saved**
6	ActivityTaskScheduled	P2 plan

…

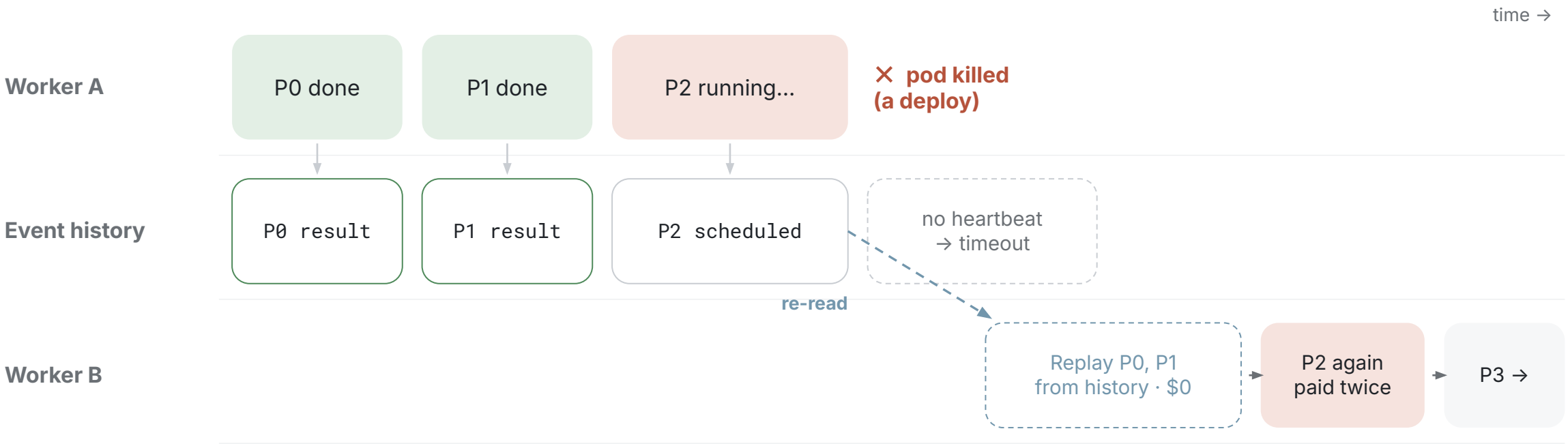
Results are stored here, not just the fact that a step ran.

If a worker dies, a new one re-reads the history and carries on.



HOW TEMPORAL WORKS

When a worker dies mid-run: replay, then carry on



Finished steps are free

Their results come from history. The code runs again, the work does not.

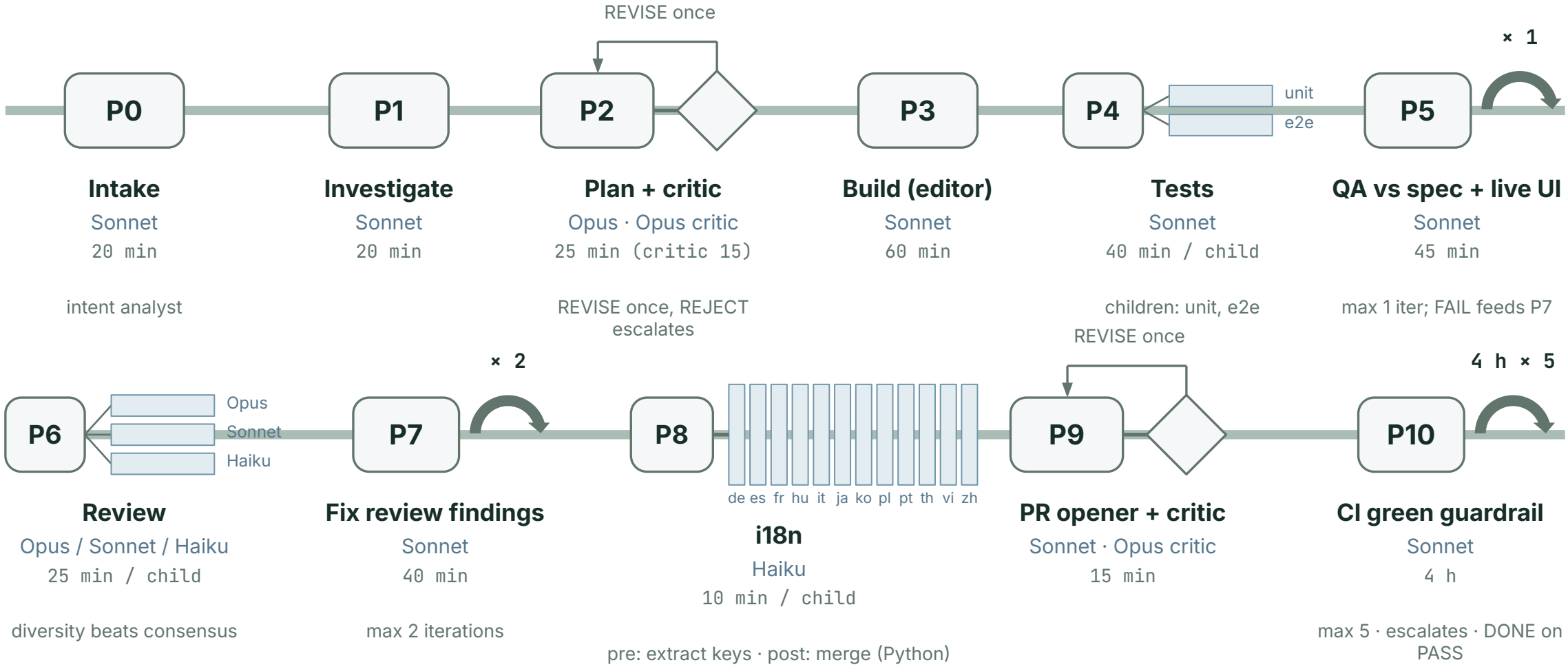
The running step is paid again

Keep paid steps small, and save each result before the next step.

Death is noticed by a timeout

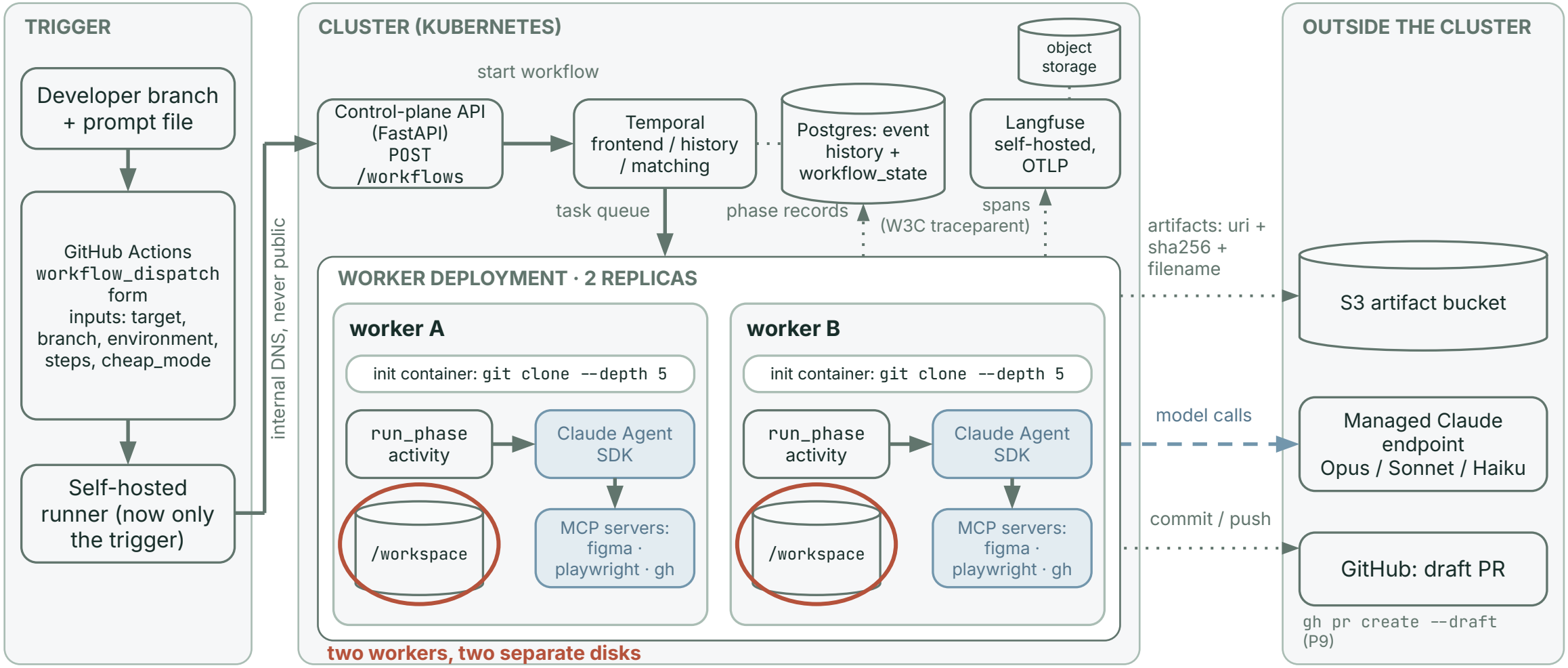
Without heartbeats, a dead worker looks alive until the timeout ends.

Eleven phases, four shapes, one for-loop

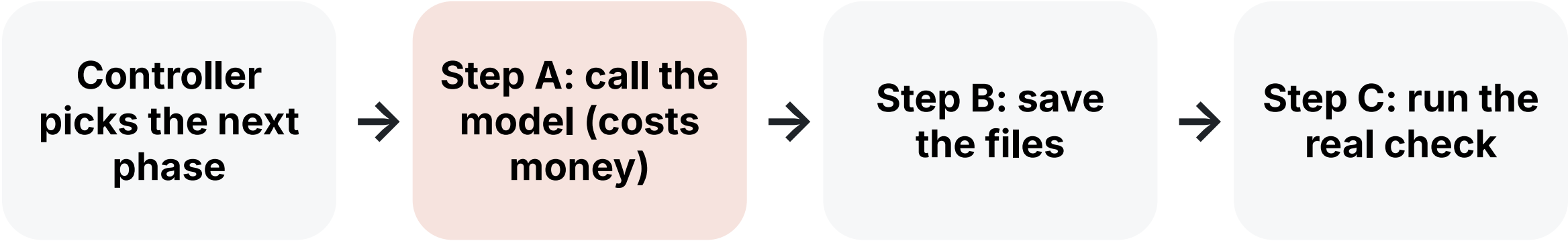



```
final_phase = "ESCALATED"  # pessimistic default
```

Everything I now have to run



The thin controller: one step to call, one step to save

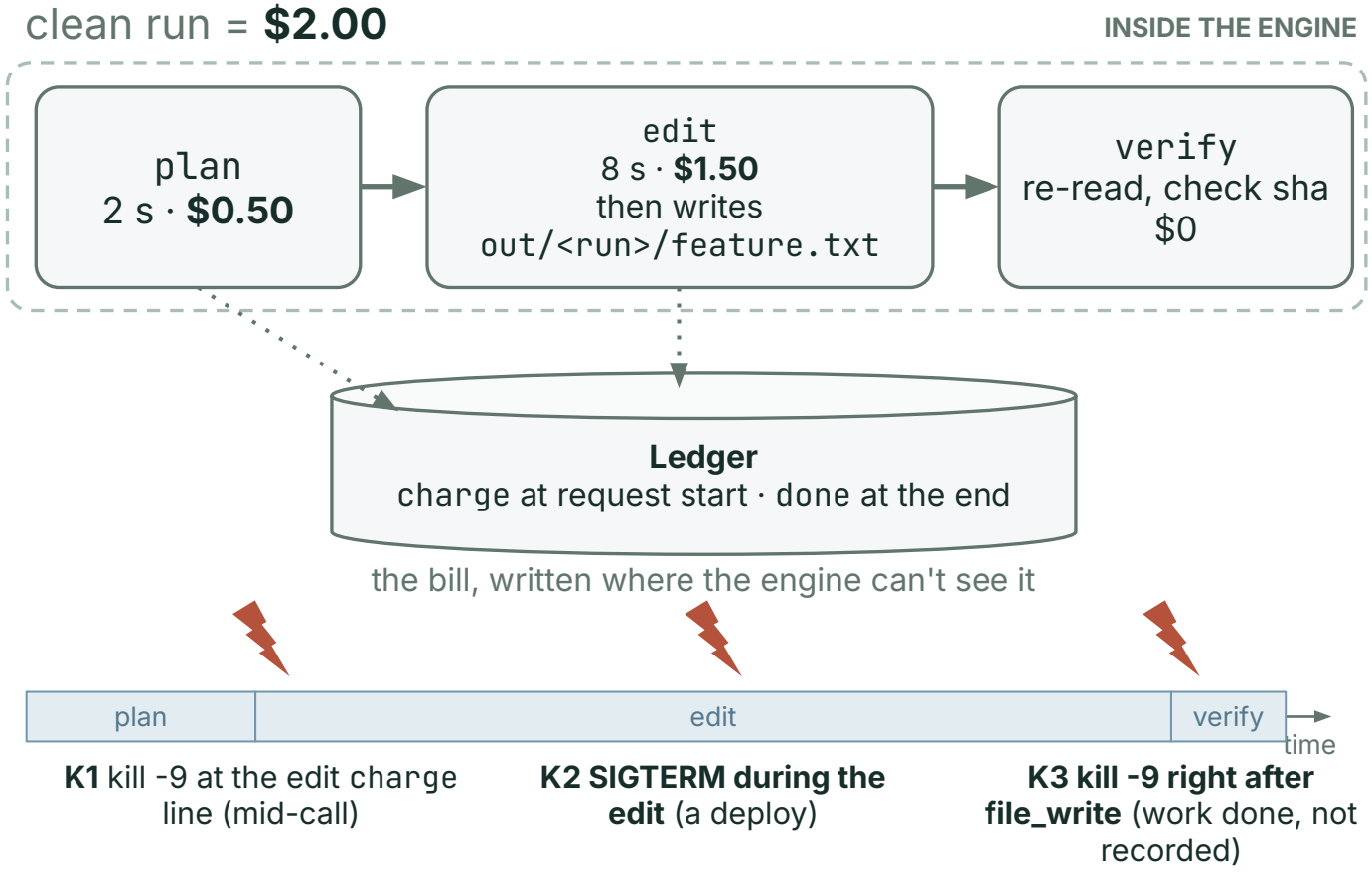


If the worker dies after the model call, the retry finds the saved result and doesn't pay again. In my code, one function, run_phase, is the only place that calls Claude.

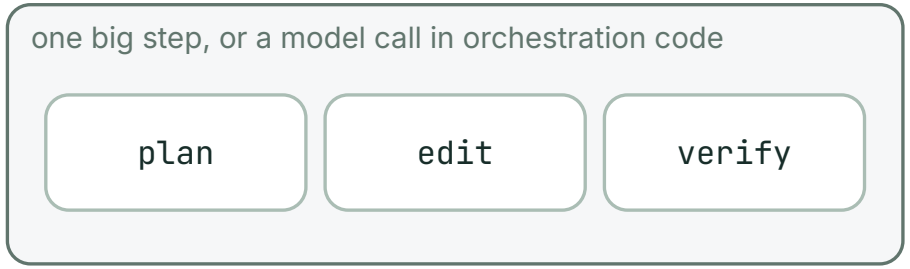
One $2.00 job, three ways to kill it, six engines

I crashed the same small job on purpose, on six engines. A fake model, with no AI and fixed prices, logs every charge to a bill the engine can't see.

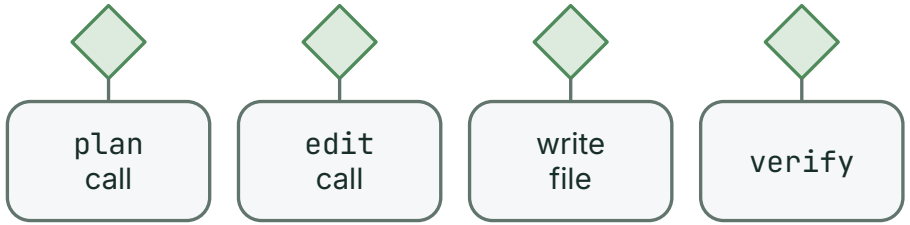
clean run = **$2.00**



Variant A natural but wrong



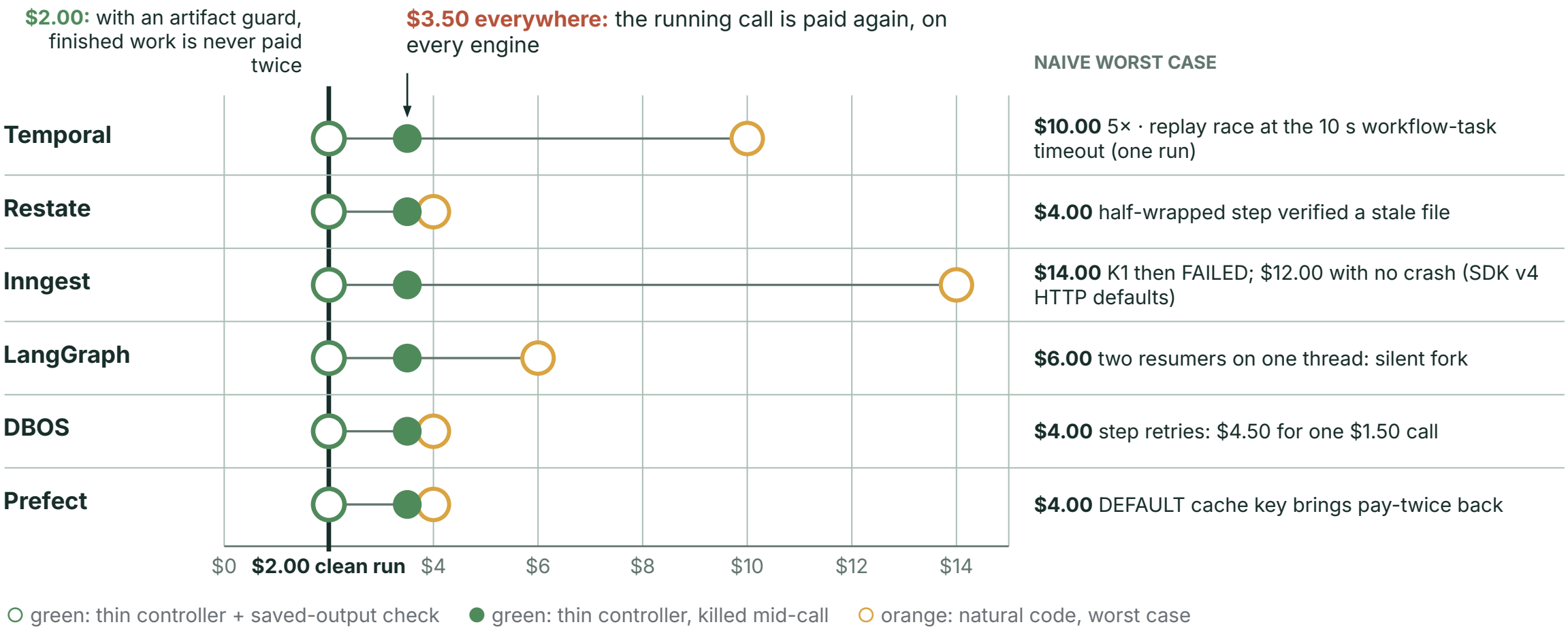
Variant B thin controller



Each model call and each file write is its own step. ◆ = skip if `out/<run>/<step>` already has the expected sha.

ENGINES: Temporal · Restate · Inngest · LangGraph · DBOS · Prefect

Same job, $2 to $14: the shape of the code set the bill



Every engine paid twice when the model call and the file write shared one step.

The engine said OK. The bill said no.

Temporal

ENGINE SAYS
✓ one clean activity
`the retry only shows up in the JSON output`

≠

LEDGER
$10.00 / $4.00

Restate

ENGINE SAYS
✓ completed with success
own `cost` output: $2.00

≠

LEDGER
$3.50

Inngest

ENGINE SAYS
✓ COMPLETED, each step once

≠

LEDGER
$6.00
classic mode, no crash

LangGraph

ENGINE SAYS
✓ no error in 34 runs

≠

LEDGER
$6.00
with two resumers

DBOS

ENGINE SAYS
✓ steps list empty
workflow's own cost: $2.00

≠

LEDGER
$3.50

Prefect

ENGINE SAYS
✓ every task Completed

≠

LEDGER
$4.00

Count your money outside the engine. The engine costs cents per run. The model costs dollars.

04

PART FOUR OF FIVE

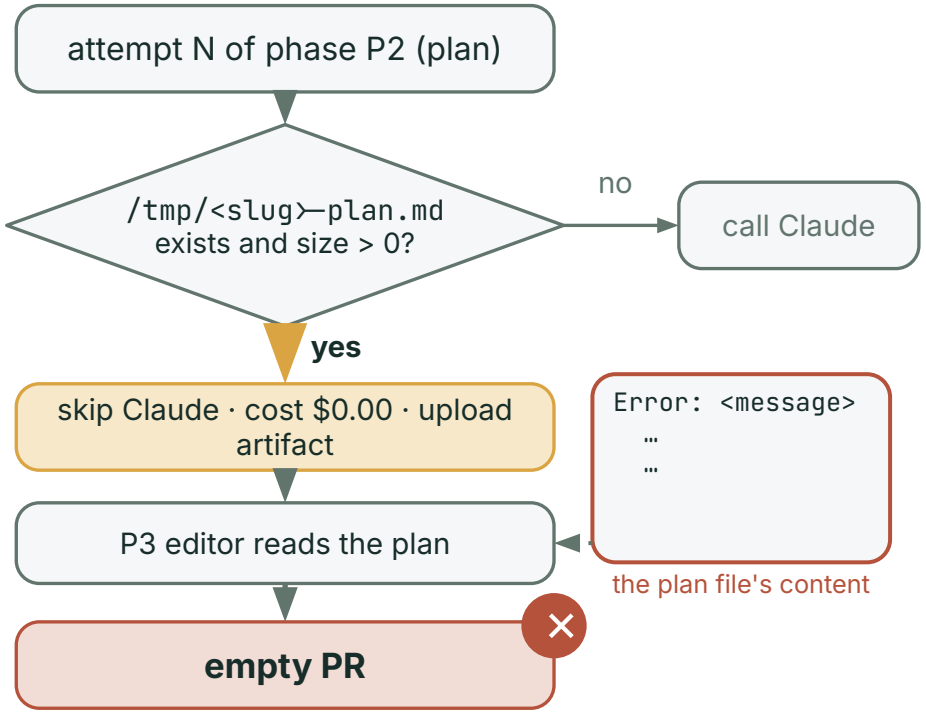
What broke

War stories, and the lessons they taught me

IDEMPOTENCY

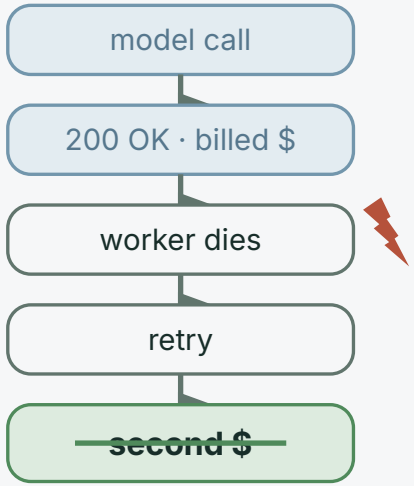
My skip-if-done check saved money, then shipped an empty PR

AS BUILT



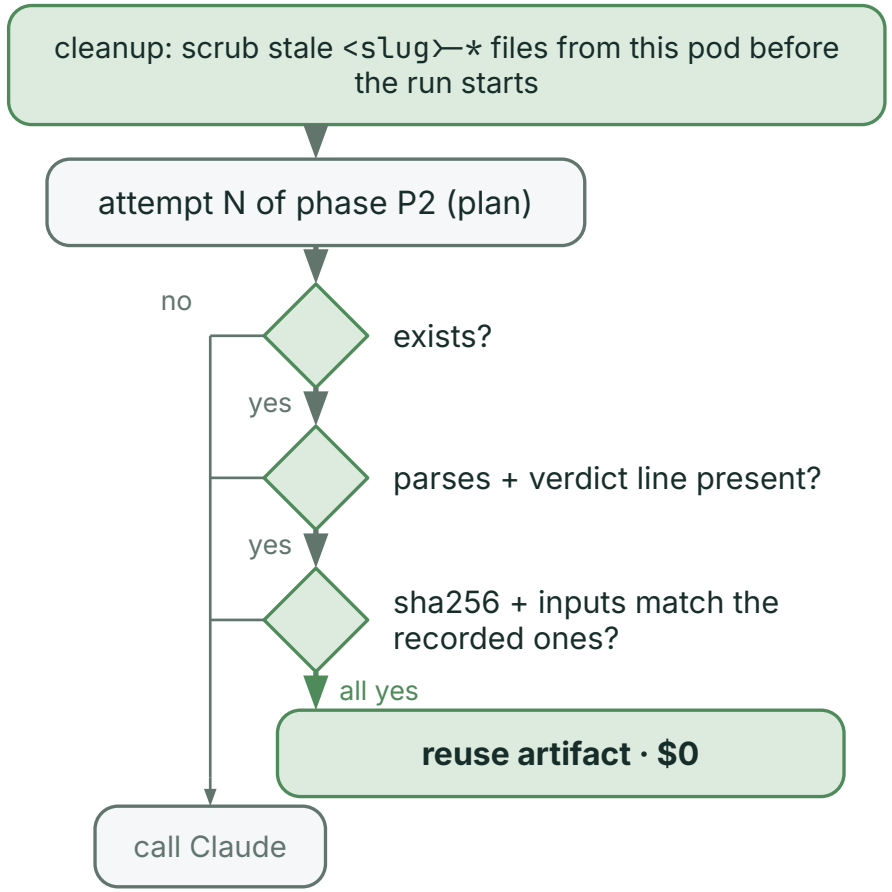
WHY IT EXISTS

A crash **after** a successful model call is the expensive one. The provider bills the 200 OK; the activity retries.



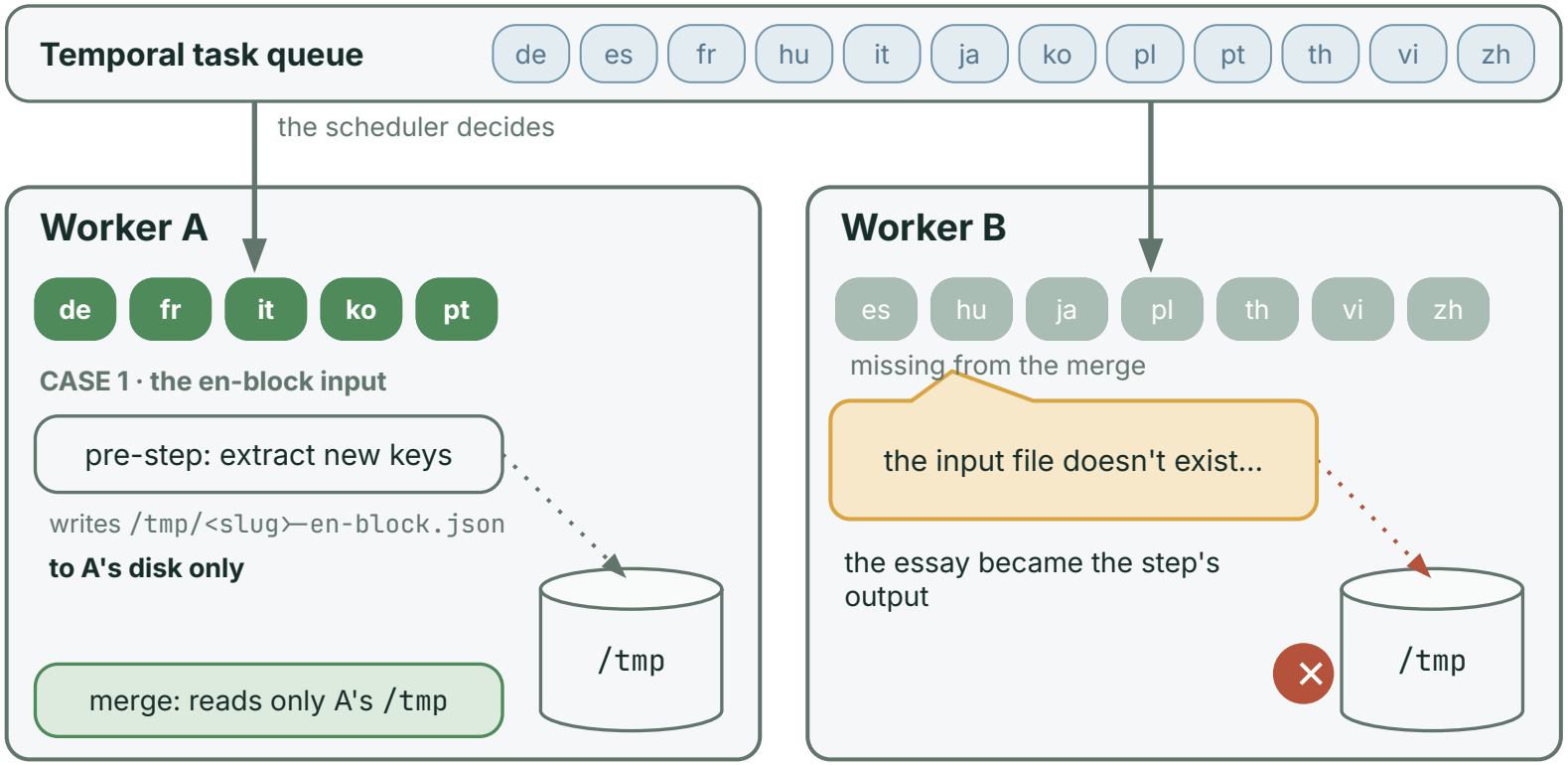
fixed path: the guard skips the paid-for work

THE FIX



LOCAL DISK

The randomness came from the scheduler, not the LLM

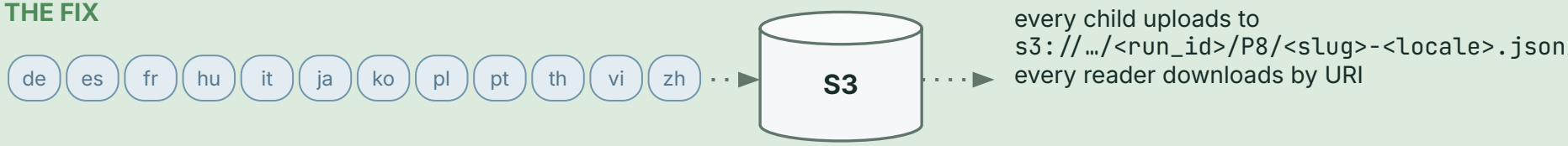


CASE 2 · THE MERGE

7 of 12

locale blocks reported missing; the merge saw only the **5** written on its own pod

THE FIX



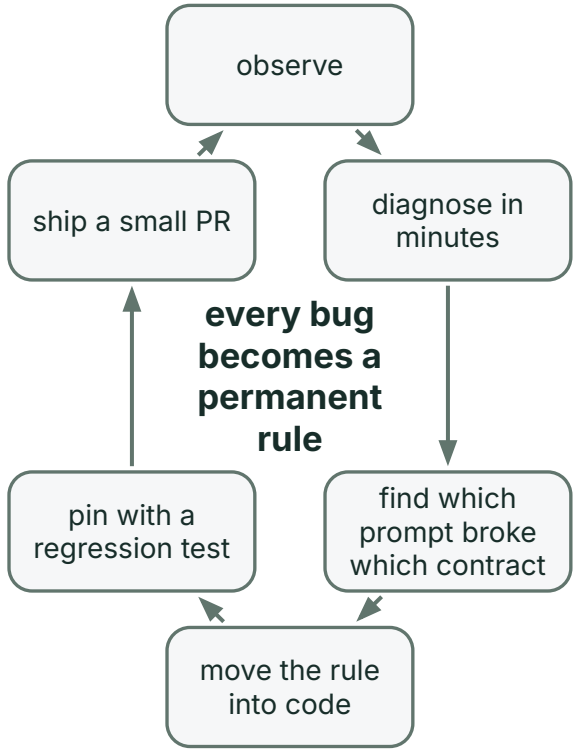
184/184

keys validated by Python

HARNESS CONTRACTS

Eight bugs, each found in minutes, and none was the model's fault

#	Bug	Contract that broke	Found in	Fault side
1	Intake punted a design conflict as an "open question"	intake must resolve or escalate (prompt contract)	S3 artifact	**harness** model infra
2	Translations all English	file-name contract between prompt and worker	S3 artifact	**harness** model infra
3	Each fan-out child guessed a different JSON root path	output schema owned by code	S3 artifact	**harness** model infra
4	A **1.3-second race** ran the diff before the branch switch	ordering between git steps	Temporal UI + logs	**harness** model infra
5	UI shipped raw i18n keys	four gates	rendered UI	**harness** model infra
6	CI phase: 10 commits and $10 by push-and-poll	CI used as test runner	git history + Langfuse cost	**harness** model infra
7	Pod SIGTERM'd mid-deploy, reported as a heartbeat timeout	grace < drain	Temporal UI	harness model **infra**
8	QA skipped visual criteria (Playwright MCP tools missing from its allowed tools) and reported success	preflight: required tools exist	S3 artifact + Langfuse trace	**harness** model infra



7 of 8 harness · 1 infrastructure · 0 model

THE LIE GALLERY

Eight claims that weren't true, and what caught them

WHAT WAS CLAIMED	I18N
`"complete" ×12 translators`	

WHAT WAS TRUE

6 of 12 files still English

Python merge

WHAT WAS CLAIMED	EDITORS
`editor reports, detailed`	

WHAT WAS TRUE

0 tool calls, no diff

diff check

WHAT WAS CLAIMED	QA
`"QA: PASS"`	

WHAT WAS TRUE

CI failed 24 unit tests

CI

WHAT WAS CLAIMED	PR TEXT
`PR description: 12 files listed`	

WHAT WAS TRUE

12 files not on the branch

Opus critic

WHAT WAS CLAIMED	TYPECHECK
`"typecheck: PASS"`	

WHAT WAS TRUE

never ran; 5 invented component APIs

build gate

WHAT WAS CLAIMED	FILE COUNT
`"33 files created"`	

WHAT WAS TRUE

33 stubs

opening the files

WHAT WAS CLAIMED	ARTIFACT
`artifact saved`	

WHAT WAS TRUE

the agent's narration, saved as the JSON

downstream parse

WHAT WAS CLAIMED	METRICS
`dashboard: "0 tool calls"`	

WHAT WAS TRUE

55

raw request dumps

SELF-REPORT

Check what the agent did, not what it says

agent's report	`typecheck: PASS`
tool calls actually made	`Read` `Edit` `Edit` `Write` `Read` `Edit` `Write` …
	`Bash: pnpm typecheck` — never called
workspace truth	5 compile errors · 4 files · invented component APIs ✕

THE FIX: A GATE INSIDE THE ACTIVITY

Claude finished →

```
gate = await run_ui_build_gate(phase, workspace)   # pnpm typecheck (+ pnpm build on P9)
# on failure: ApplicationError(type="UIBuildGateFailed", non_retryable=True)
```

→ upload artifact

prompt-only requirements are not enough.

EVIDENCE

Done has to rest on real proof, not on the agent's word

honest gap: about 1 of 4 features merged after heavy human rework

DONE = P10 PASS (CI green)

8 Human review + merge

7 CI green on the pushed branch

6 Build / typecheck run by code

5 Diff is non-empty + tool calls > 0

4 Artifact parses + has its verdict token

3 Artifact exists

2 Agent's narrative (PR description, summary)

1 Agent's self-report ("PASS", "complete")

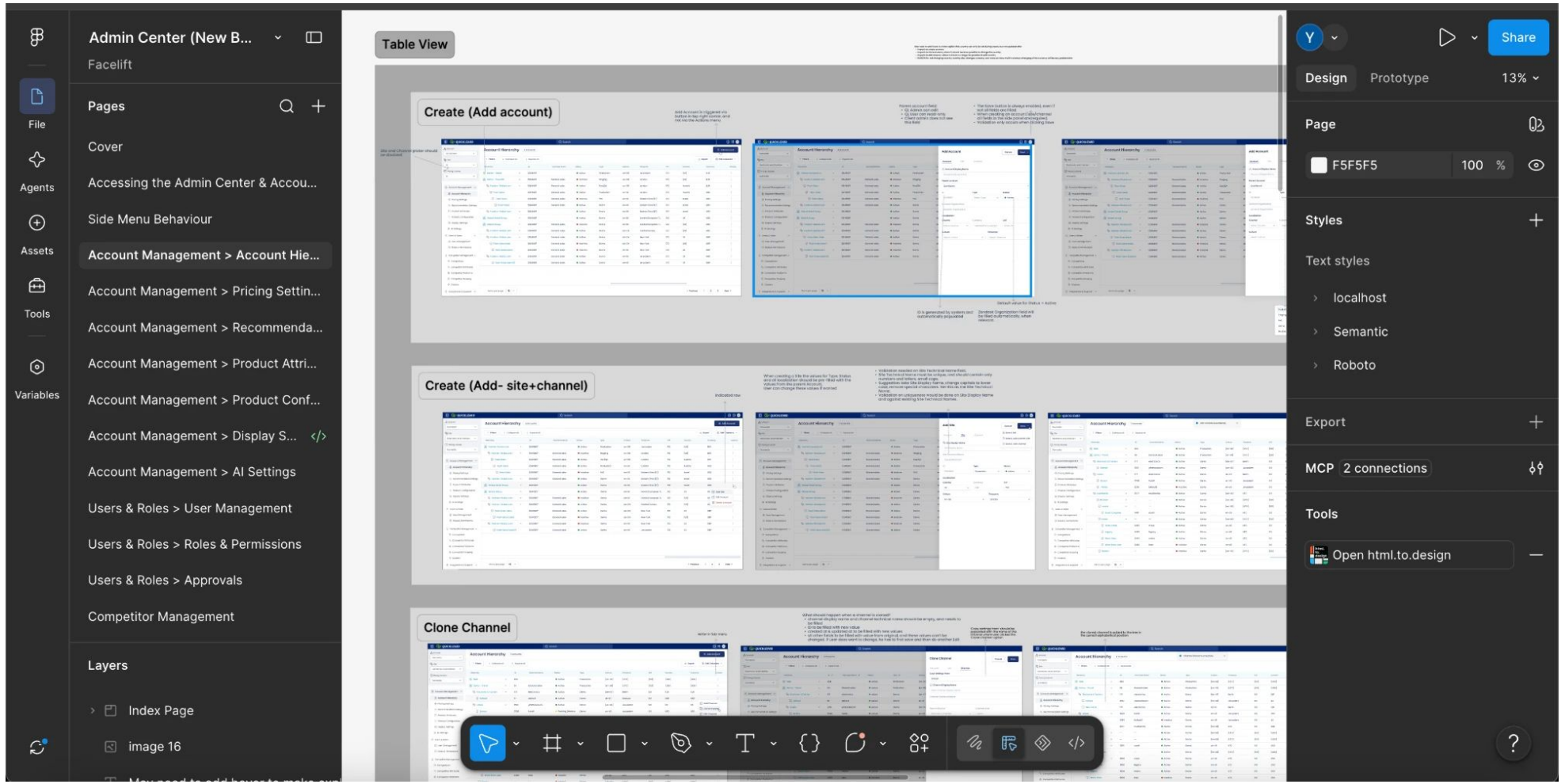
QA — QA PASS, then CI failed 24 unit tests

fidelity — side rung: fidelity (Playwright vs acceptance criteria) — 33 stubs

editors — a report, but 0 tool calls and no diff

typecheck — 'typecheck: PASS' (rung 1), caught by the build gate (rung 6)

My agent got 50+ screens, and no story

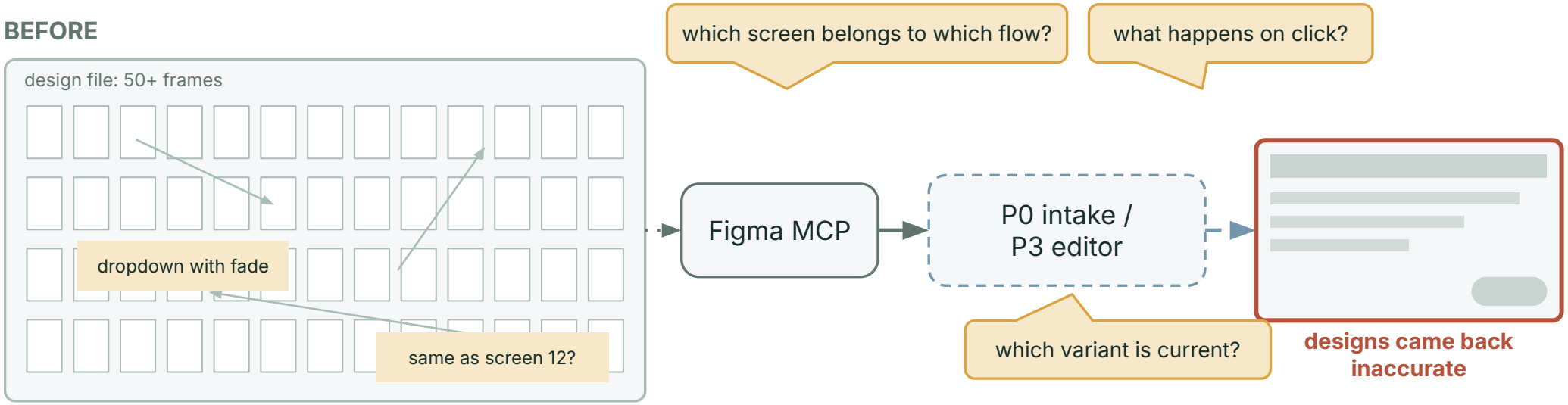


Screens, variants and notes next to them. No flow: which screen leads where, or what happens on click.

DESIGN HAND-OFF

My agents couldn't read our designers' hand-off

BEFORE



AFTER (IN PROGRESS)



the hand-off changes, not only the coding

Not fully solved; in progress.

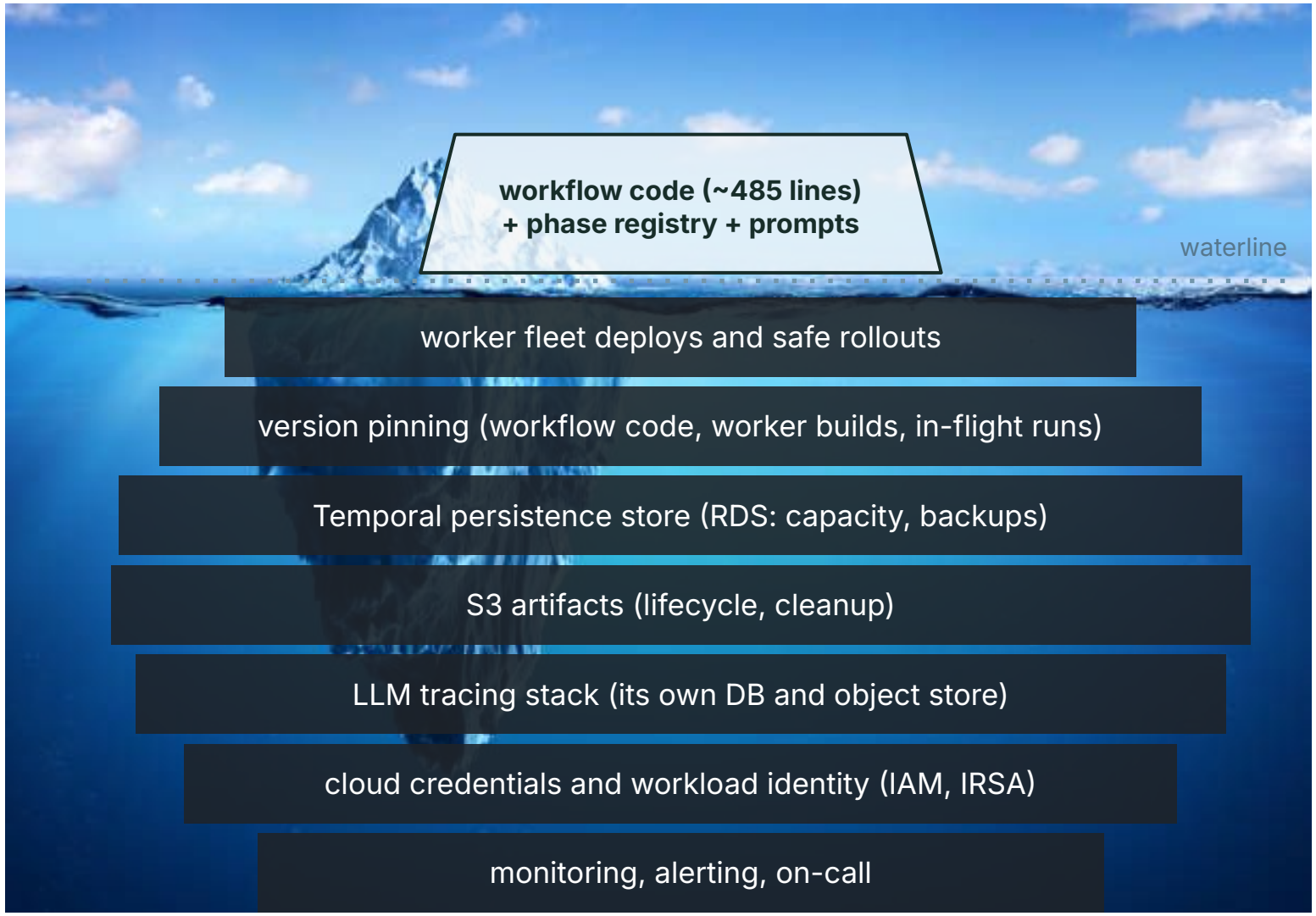
SDLC



the bottleneck moved upstream

OPERATIONS

Writing the workflow was a third of the job. Running it was the rest.



THE OPERATIONAL DRIP

- expiring SSO sessions
- VPN/DNS flakiness reaching the cluster
- model-provider identity federation broke after a platform change
- a managed-DB password rotation desynced and crashlooped a service

None of these involved an agent.

05

PART FIVE OF FIVE

Is it worth it?

What Stripe and Spotify learned, and where this is going

Scope decides who stays at the keyboard

Pair session

Developer and Claude, both in the loop

`explore → decide → redirect`

Architecture changes
Ambiguous scope
Large, iterative work

One-shot agent

Hands off until the PR is ready

`prompt → devbox → CI → PR`

Feature flags, one-line changes
On-call bug fixes
Features scoped out in advance

Context emerges while you work ⟷ **Context fits in the prompt**

stripe

Source: Mark Doyle, Stripe · Inside minions

Stripe landed in the same place

In the prompt
`“PLEASE run the linters before you commit.”`

→

In code
`lint → test → commit, in a loop`

“...deterministic code and a loop increases the success rates of these things exponentially.”

Mark Doyle, Stripe · Building autonomous coding agents at Stripe

That's the thin controller, at Stripe's scale.

stripe

Spotify's Honk: a small toolbox and strict checks

WHY THEY BUILT IT

Fleet-wide migrations ran on scripts, and the scripts got huge. One Maven updater grew past **20,000 lines**.
Now a prompt does that job.

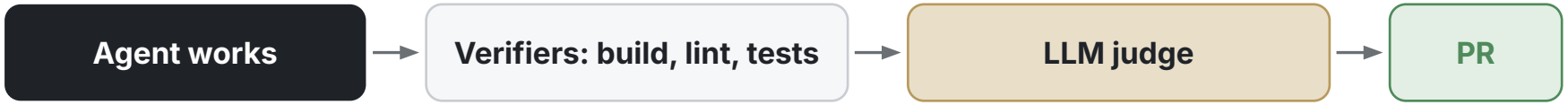
A SMALL TOOLBOX

Claude Agent SDK in Kubernetes pods. Just **three tools**: verify, a git tool that never pushes, and allow-listed Bash.
"The more tools you have, the more dimensions of unpredictability you introduce."

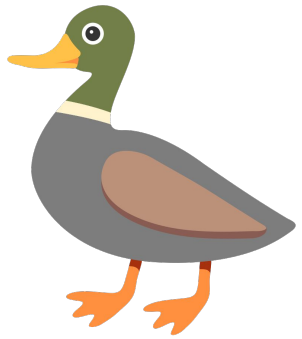
STRICT CHECKS

Verifiers run in a stop hook: if one fails, **no PR opens**.
Then an LLM judge reads the diff against the original prompt.

EVERY SESSION



Vetoes about 1 in 4 sessions · the agent fixes half itself
#1 reason: it did more than asked (refactors, disabled flaky tests)



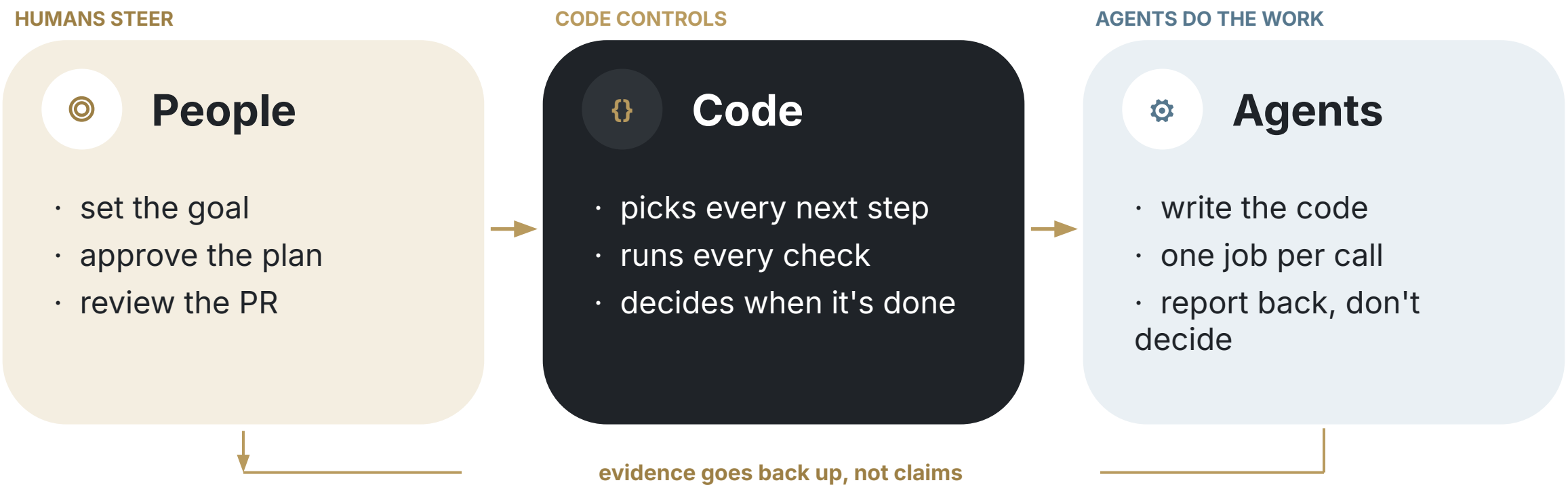
1,500+
merged agent PRs

60–90%
less time on migrations

+76%
more PRs to review, across all of Spotify

Spotify®

People set the goal, code runs it, agents do the work



Talk behind this: **Harness Engineering**, Ryan Lopopolo, OpenAI.
His split matches what I ended up building.

OpenAI

IS IT WORTH IT?

The biggest teams are all building a software factory

Stripe
Minions

1,000+ merged PRs a week with no human-written code, all human-reviewed

Spotify
Honk

10 days to 1,000 merged agent PRs. It used to take 3 months.



Ramp
Inspect

75% of merged PRs come from agent sessions

Uber
cloud agents

70%+ of PRs are made through coding agents

Coinbase
parallel agents

95%+ of code written by or with LLMs

Shopify
River

1 in 8 merged PRs co-authored by the agent

Block
Builderbot

15% of production changes made almost fully by agents

DoorDash
Flux

130K automated engineering tasks in one month

Intercom
Claude Code + skills

2× merged PRs per engineer

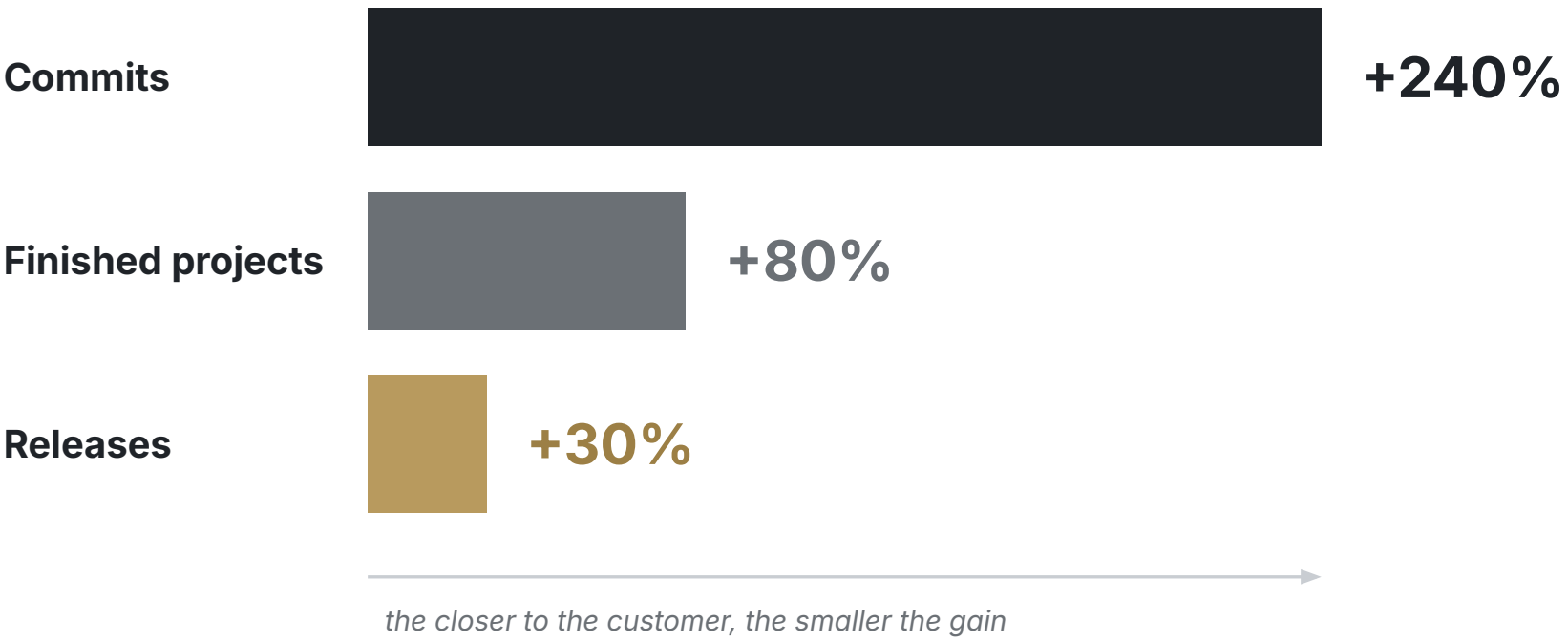
Each company counts differently. Compare the direction, not the numbers.

On all of GitHub, 1 in 3 pull requests now involves an agent. Satya Nadella, Microsoft CEO

IS IT WORTH IT?

More code, not much more shipping

Developers using autonomous coding agents, 500,000+ on GitHub (NBER study)



HARVARD STUDY · 718 COMPANIES

Coding productivity

+20–30%

Shipped work

no significant gain

Engineers spend only 30–40% of their time writing code.

Chen & Stratton, Harvard (2026)

Code got faster. Shipping did not keep up.

IS IT WORTH IT?

The new bottleneck is everything around the code

Harvard study

+49%

review time after companies adopt agents

Chen & Stratton, Harvard · 718 companies

Spotify

+76%

more pull requests to review

Spotify engineering

ANTHROP\C

~80%

of an agent session is spent waiting for a human

Anthropic · vendor data

Stack Overflow

1 in 5

developers use AI to deploy or run production

Stack Overflow developer survey



The factory makes parts fast. The line around it still runs at human speed.

Every burn became a guardrail

	THE BURN	THE GUARDRAIL
Spotify	Agents commented out failing tests to make the build green.	**Verify the change before a PR can open.**
	An agent deleted a production database during a code freeze.	**Separate dev and prod. A planning-only mode.**
ANTHROP\C	Long-running agents said the job was done too early.	**Start from failing features. Agents may only flip one to passing.**
MY PIPELINE	It stopped at phase 8 and still reported success.	**Code owns the plan. A missing step is an error.**
MY PIPELINE	Agents wrote “typecheck passed” without running it.	**Code runs every check and reads the exit code.**

“Anytime you find an agent makes a mistake, you take the time to engineer a solution.”

Mitchell Hashimoto, co-founder of HashiCorp

IS IT WORTH IT? · MY ANSWER

My answer: not yet, but close

NOT YET

- About 1 in 4 features merged
- $15–25 per run
- People still review everything

ALREADY USEFUL

- Boring phases run without me
- I can read every step afterward
- Real checks catch false claims



Engineers build the factory, not the features

THE VISION

where I believe our job is heading

Features
the agents build these

ENGINEERS OWN THESE

Agents

ANTHROP\C agent work spent building for other agents: 10% → 22%

Harness
checks, skills, playbooks

Uber 2,500 skills

DOORDASH 300+ playbooks

MCP servers and tools

Uber 1,000+ tools, one gateway

stripe 400+ tools

Company brain
shared context

Uber 40M-entry context graph

ANTHROP\C 8,500 agent instructions

"They build the software factory rather than the software." PostHog

IS IT WORTH IT? · MEASURING THE FACTORY

Uber puts a price on every unit of agent work

Uber

	Code gen	Validate	Deploy	Observe	Maintain	skill	cloud
SPECIALIZED AGENTS agent-driven, human in the loop, on Uber compute	**Minion** intent → PR	**uReview** PR → code review	**Agentic XP** XP done → readout	**Conan AI** alert → root cause	**Fawkes** cron → maintenance	●	●
	cost / **merged PR**	cost / **review**	cost / **XP readout**	cost / **alert**	cost / **cleanup**		
GENERAL AGENTS interactive, on Uber compute	**Cortana: a zero-setup agent that runs any skill**				cost / **query**	●	●
SESSIONS WITH SKILLS interactive, on your laptop	**Agentic tasks with 3,600+ shared skills**				cost / **session**	●	○
RAW SESSIONS interactive, on your laptop	**Claude Code, Codex, OpenCode: you bring the context**				cost / **session**	○	○

Narrowest task → a benchmark from real work → the Pareto-optimal model.
Every dollar maps to a unit of work.

The narrower the agent, the easier it is to measure, and the more you control its cost and model.

● skill: a continuously improved skill set ● cloud: runs on Uber compute ○ neither

Source: Uber Engineering blog

Five checks before you pick an orchestrator

None of them depends on the tool you pick.

1. Let **code** pick the next step, not the model
2. Keep the model call **in its own step**
3. Skip finished work only if its **saved output is valid**
4. Run the checks yourself and **read the exit code**
5. Count what each run costs, **outside the engine**

The model writes the code.
Your code decides when it's done.

Pick the shape first. The tool second.

Thank you

I'd love to hear your questions, war stories, or your own orchestrator scars :)

Yedidya Schwartz
CTO, Quicklizard

All the resources from this talk
github.com/yedidyas/ai-sdlc

Scan for all my links
linktr.ee/yedidyas